JTC Work Experience

JSC "JT Consulting", Russia, Saint Petersburg

jtc.ooo

Work Periods

Oct 2019 – Aug 2025
Senior Frontend Developer

Oct 2018 – Oct 2019
Frontend Developer

CODELESS CMS System

CODELESS Expression Editor Development

Innovative code editor with DOM architecture and full typing.

GOAL

Create tool for writing pseudocode (JavaScript-like) that will later be interpreted to JavaScript and executed on client. Code writing is necessary when creating CODELESS projects. This tool should accelerate project creation process.

FEATURES

- Smart suggestion generation when writing code
- Full real-time error validation
- Using DOM elements instead of text representation
- Special attention to performance
- Full typing and context passing
- Adaptivity for different screens
- Font scaling
- Full Copy/Cut/Paste operations support

ERROR VALIDATION

- Real-time syntax validation
- Highlighting erroneous code sections

DOM ARCHITECTURE

- Complete rejection of text representation
- Each element is separate DOM element

- Detailed error messages
- Type and context checking

TYPING

- TypeScript typing at all levels
- Automatic type detection
- Context hints
- Type compatibility checking

- Interactive controls
- Complex focus management system

OPTIMIZATION

- "Only what's needed" rendering system
- Code element formatting calculation system (+ caching)
- Calculation memoization

React TypeScript

Configuration Comparison System

Tool for comparing different versions of project configuration files

- Real-time calculations (on user demand)
- String comparison algorithm (all projects can be represented as string data)
- Full Web Workers operation for optimization (doesn't block main thread)

TypeScript Web Workers

DEMO

Usage Search Mechanism

Automatic calculation of all usage places for any project element

- On-the-fly calculation each time on user demand
- Web Workers operation
- Fast navigation to any CODELESS application part
- Default page search replacement

TypeScript Web Workers Fast Diff

DEMO

Project Integration System

Integration/updating projects from other CMS instances to avoid code duplication

- Link system to prevent changes
- Update-only capability
- Conflict detection

Node.js TypeScript

DEMO

Module Connection System

Connecting modules from any repositories without storing in common bundle

- Support for various repositories (NPM, Nexus)
- Direct repository API requests
- Installation to server file system
- Using npm cli on server

Node.js TypeScript NPM/Nexus

DEMO

Migration from File System to RavenDB

Comprehensive architecture migration to solve performance and scalability issues.

PROBLEM

Previous architecture stored project configs in JSON files, causing performance issues with large projects. Memory filled up quickly, read/write speed was suboptimal.

SOLUTION

Together with technical director, decided to migrate to RavenDB (NoSQL with out-of-the-box transaction support). Participated in migration and backend rewrite from scratch.

COMPLETED

- Complete migration from file system to RavenDB
- Backend architecture rewrite
- Transaction system implementation
- Performance and memory optimization
- Scalable architecture for large projects

Node.js RavenDB Express TypeScript

Automatic "Pod" Launch System

Automation of deployment and management of individual project instances.

PROBLEM

DevOps manually configured servers for each project version and wrote nginx config rules. Process was slow and error-prone.

SOLUTION

Developed automatic creation and launch system for separate child processes in Node.js with automatic port and proxy management.

COMPLETED

- Automatic creation and launch of separate child processes in Node.js
- Each project version runs on separate server with unique port
- User gets link with port and sees real working application (config build)
- Developed proxy configuration system for each pod (redirection, header changes, HTTP/WS connections, timeouts, path replacement rules)

Node.js Child Processes Express Proxy TypeScript

SSE Communicator

Typed client-server interaction system using Server-Sent Events.

PROBLEM

Need for transparent and typed API interaction between client and server with real-time update support.

SOLUTION

Created SSE communicator using Proxy class in Node.js to hide HTTP method addresses and provide full API typing.

COMPLETED

- Technology: Server-Sent Events (SSE)
- Architecture: TypeScript typed HTTP methods (GET, POST, Channel)
- Feature: Using Proxy class in Node.js to hide HTTP method addresses
- Syntax: await protectedApiClient.user.GET.byId({ id: "123" })
- Benefits: Transparent client-server interaction with full typing

TypeScript | Server-Sent Events | Node.js | Proxy API

Other JTC Projects

React Native Module for PayControl

Seamless integration with PayControl software for document signing from mobile devices.

React Native | TypeScript | PayControl SDK

UI-KIT Development and Support

- Creating intra-platform components
- Refactoring existing solutions
- Library support and enhancement

React | TypeScript

Code Documentation Tool

Process documentation automation with mandatory updates.

How it works:

- Forces documentation writing
- CI checks actuality on code updates
- Interrupts review and deploy if outdated

React | TypeScript | TypeDoc

Banking System Experience

Banking System Module Development

Comprehensive module development for corporate banking systems with high security and functionality requirements.

Clients: VTB, Gazprom, Otkritie

IMPLEMENTED MODULES

Account module | Statement module | Payment order module | Employee management module | Bank card module | Admin panel

TECHNICAL FEATURES

- Complex forms with multiple validation (frontend + backend)
- Multi-step forms with intermediate saves
- Tables with large data volumes (scrollers)
- Modal windows and directories
- Backend error processing and beautiful display

React | Redux | Redux Form | TypeScript | CryptoPro

Banking Page Builder Helper

Development automation tool for accelerating typical banking system page creation.

PROBLEM

Banking system page development required much time due to repetitive patterns and lack of unified code style. Each page had to be written from scratch, slowing development process.

SOLUTION

Created automatic generation tool for typical banking pages with unified patterns and code style. Tool allows quick creation of basic page structure with ready components.

COMPLETED

- Automatic generation of typical banking pages
- Architecture and code style unification
- Ready templates for main components
- Development process acceleration by several times
- Error reduction through standardization

React | TypeScript

Technology Stack and Expertise

Main Technologies

Frontend:

React Redux TypeScript

Backend:

Node.js Express

Mobile:

React Native

Databases:

RavenDB

Expertise

- Performance optimization (Web Workers, processes)
- State management systems
- Databases and repositories
- Transaction systems
- Responsive layout and cross-browser compatibility
- Cross-platform development (Web, Mobile)
- Visual coding and interactive development tools
- High-load enterprise applications

AI/ML Technologies

- Language model fine-tuning
- Ollama
- LoRA
- Docker containerization
- Process automation

APIs & Tools

- REST, WebSocket, SSE
- Telegram Bot API
- Webpack
- Web Workers